

Proj4: 数据库安全与完整性

1 实验目的

本实验基于 Project1 已经完成的 Olist 电商 3NF 数据库，围绕数据库安全性与完整性控制展开。实验重点不是应用层简单做输入判断，而是将关键业务约束和访问控制规则下沉到数据库内核中，通过触发器、视图和用户授权机制完成统一约束。

本实验主要完成以下目标：

1. 基于电商订单明细表定义一条复合业务规则，并使用触发器在数据库层强制执行。
2. 选择一个敏感字段，通过脱敏视图向普通分析师隐藏该字段。
3. 创建受限数据库用户，并通过 `GRANT` / `REVOKE` 实现自主存取控制。
4. 使用 Streamlit 构建具备角色切换、权限验证和完整性验证功能的交互界面。
5. 在 Python 中捕获数据库返回的异常信息，验证完整性违规和越权访问均由数据库层拦截。

2 实验环境

项目	配置
数据库	openGauss
数据库名	ecommerce_db
Schema	proj1_3nf
开发语言	Python
数据库连接库	psycopg2
可视化框架	Streamlit
数据来源	Project1 导入并规范化后的 Olist 电商数据

Project4 直接复用 Project1 中已经完成的 3NF 表结构，其中与本实验关系最密切的是 `order_items` 和 `products` 两张表：

```
1 CREATE TABLE order_items (  
2     order_id VARCHAR(32) NOT NULL,  
3     order_item_id INTEGER NOT NULL CHECK (order_item_id > 0),  
4     product_id VARCHAR(32) NOT NULL,  
5     price NUMERIC(10, 2) NOT NULL CHECK (price >= 0),  
6     freight_value NUMERIC(10, 2) NOT NULL CHECK (freight_value >= 0),  
7     PRIMARY KEY (order_id, order_item_id),  
8     FOREIGN KEY (order_id) REFERENCES orders(order_id),  
9     FOREIGN KEY (product_id) REFERENCES products(product_id)  
10 );
```

分析：`order_items` 表中包含商品售价 `price` 和运费 `freight_value` 两个数值字段。二者共同描述了一个订单项的交易成本结构，因此适合用于设计涉及多个字段的业务完整性规则。

3 业务规则定义

3.1 完整性红线

本实验定义的业务红线为：

订单项的运费 `freight_value` 不能高于商品售价 `price`。

对应的逻辑判断为：

```
1 | freight_value > price
```

若插入或更新的订单项满足上述条件，则说明该订单项出现异常物流费用，数据库应拒绝该操作。

分析：在 Olist 电商场景中，`price` 表示商品售价，`freight_value` 表示该订单项对应的运费。虽然现实业务中可能存在运费偏高的情况，但在课程实验中，这一规则可以清晰地体现多字段语义约束。它不同于单列 `CHECK (price >= 0)` 这类简单约束，而是需要同时读取 `price` 和 `freight_value` 两个字段后进行判断，符合实验对复合业务规则的要求。

3.2 敏感字段

本实验选择 `freight_value` 作为敏感字段。

- `freight_value`：订单项运费，代表物流费用信息。
- 管理员：可以直接查看 `order_items` 物理表，包含 `freight_value`。
- 分析师：只能查看脱敏视图 `v_public_order_items`，不包含 `freight_value`。

分析：普通分析师通常只需要查看订单、商品类别与售价，用于进行销售分析或品类分析。物流费用属于更细粒度的成本信息，若直接暴露给所有用户，会扩大敏感数据访问范围。因此通过视图隐藏该字段，可以体现数据库在属性级访问隔离中的作用。

4 数据库完整性控制

4.1 触发器函数

完整性控制由 `check_order_item_freight()` 函数实现。该函数在每次插入或更新 `order_items` 前检查新数据行，如果 `freight_value > price`，则通过 `RAISE EXCEPTION` 中断事务。

```
1 | CREATE OR REPLACE FUNCTION check_order_item_freight() RETURNS TRIGGER AS $$
2 | BEGIN
3 |     IF NEW.freight_value > NEW.price THEN
4 |         RAISE EXCEPTION '数据完整性拦截：订单项运费 % 不能高于商品售价 %',
5 |             NEW.freight_value,
6 |             NEW.price;
7 |     END IF;
8 |
9 |     RETURN NEW;
10 | END;
11 | $$ LANGUAGE plpgsql;
```

其中：

- `NEW`：表示即将插入或更新的新数据行。

- `NEW.freight_value`: 新订单项中的运费。
- `NEW.price`: 新订单项中的商品售价。
- `RAISE EXCEPTION`: 主动抛出数据库异常，中断当前 SQL 操作。
- `RETURN NEW`: 当数据符合规则时，允许数据库继续执行插入或更新。

分析：该约束放在数据库层实现后，无论数据来自 Streamlit、Navicat 还是命令行 SQL，只要试图写入违规数据，都会被同一个触发器拦截。这比只在前端或 Python 中检查更可靠。

4.2 触发器绑定

触发器绑定到 `order_items` 表的 `BEFORE INSERT OR UPDATE` 时机：

```
1 CREATE TRIGGER trg_check_order_item_freight
2 BEFORE INSERT OR UPDATE ON order_items
3 FOR EACH ROW EXECUTE PROCEDURE check_order_item_freight();
```

分析：选择 `BEFORE INSERT OR UPDATE` 是为了同时覆盖新订单项录入和已有订单项修改两种场景。如果只检查 `INSERT`，之后仍可能通过 `UPDATE` 将原本合法的数据改成违规数据。

5 数据库安全性控制

5.1 脱敏视图设计

为了隐藏敏感字段 `freight_value`，实验中创建了视图 `v_public_order_items`。该视图保留订单项的基本分析字段，并额外关联 `products` 表得到商品类别，但不暴露运费字段。

```
1 CREATE OR REPLACE VIEW v_public_order_items AS
2 SELECT
3     oi.order_id,
4     oi.order_item_id,
5     oi.product_id,
6     p.product_category_name,
7     oi.price
8 FROM order_items oi
9 JOIN products p
10 ON oi.product_id = p.product_id;
```

视图字段如下：

字段	说明
order_id	订单 ID
order_item_id	订单项序号
product_id	商品 ID
product_category_name	商品类别
price	商品售价

分析：视图相当于为分析师提供一个受控的数据窗口。分析师仍然可以进行订单项与商品类别相关的查询，但无法从结果中获得 `freight_value`。这说明安全控制并不一定需要复制一份脱敏数据，合理使用视图即可在原始数据基础上构造不同用户的数据可见范围。

5.2 用户与权限配置

实验中创建受限用户 `analyst_user`，先收回其对 `schema` 内表的全部权限，再只授予视图查询权限：

```
1 DO $$
2 BEGIN
3     IF EXISTS (SELECT 1 FROM pg_catalog.pg_roles WHERE rolname = 'analyst_user')
4     THEN
5         REVOKE ALL PRIVILEGES ON ALL TABLES IN SCHEMA proj1_3nf FROM analyst_user;
6         REVOKE USAGE ON SCHEMA proj1_3nf FROM analyst_user;
7     END IF;
8 END $$;
9 DROP USER IF EXISTS analyst_user;
10 CREATE USER analyst_user WITH PASSWORD 'Analyst@123';
11
12 GRANT USAGE ON SCHEMA proj1_3nf TO analyst_user;
13 REVOKE ALL PRIVILEGES ON order_items FROM analyst_user;
14 GRANT SELECT ON v_public_order_items TO analyst_user;
15 ALTER USER analyst_user SET search_path TO "$user", proj1_3nf, public;
```

分析：`GRANT USAGE ON SCHEMA` 只允许用户进入该 `schema` 查找对象，并不等价于授予表访问权限。真正的数据访问权限由 `GRANT SELECT ON v_public_order_items` 提供。由于没有授予 `order_items` 的查询权限，分析师即使知道物理表名，也无法直接查询基表。

6 Streamlit 应用实现

6.1 代码整体框架

`project4_app.py` 的整体结构分为三个部分：

1. 数据库连接配置与角色定义。
2. 数据库访问工具函数。
3. Streamlit 交互界面，包括权限测试与触发器测试。

核心角色定义如下：

```
1 ADMIN_ROLE = "管理员 (Admin)"
2 ANALYST_ROLE = "分析师 (Analyst)"
3
4 SCHEMA = os.getenv("DB_SCHEMA", "proj1_3nf")
5 BASE_TABLE = "order_items"
6 PUBLIC_VIEW = "v_public_order_items"
7 SENSITIVE_COL = "freight_value"
```

分析：通过常量统一保存物理表、视图和敏感字段名称，可以避免后续界面逻辑中反复硬编码字符串。管理员和分析师角色在前端通过单选按钮切换，在后端对应两套不同的数据库连接配置。

6.2 角色连接切换

数据库连接函数如下：

```
1 def get_connection(role_name):
2     """根据当前会话身份建立数据库连接，并进入 Project1 的 3NF schema。"""
3     config = ADMIN_CONFIG if role_name == ADMIN_ROLE else ANALYST_CONFIG
4     conn = psycopg2.connect(**config)
5     with conn.cursor() as cur:
6         cur.execute(
7             pg_sql.SQL("SET search_path TO {}, public;").format(
8                 pg_sql.Identifier(SCHEMA)
9             )
10        )
11    return conn
```

分析：角色切换并不是在应用层简单改变显示内容，而是重新使用不同数据库用户建立连接。管理员使用

`DB_USER`，

分析师使用 `ANALYST_DB_USER`。

因此，当分析师执行越权 SQL 时，请求会直接以 `analyst_user` 身份提交给数据库，由数据库权限系统决定是否允许访问。

6.3 数据浏览模块

管理员查询物理表，并显示敏感字段：

```
1 ADMIN_QUERY = """
2     SELECT
3         oi.order_id,
4         oi.order_item_id,
5         oi.product_id,
6         p.product_category_name,
7         oi.price,
8         oi.freight_value
9     FROM order_items oi
10    JOIN products p
11      ON oi.product_id = p.product_id
12   ORDER BY oi.order_id DESC, oi.order_item_id DESC
13   LIMIT 200;
14 """
```

分析师查询脱敏视图：

```
1 ANALYST_QUERY = """
2     SELECT *
3     FROM v_public_order_items
4     ORDER BY order_id DESC, order_item_id DESC
5     LIMIT 200;
6 """
```

在界面展示后，程序根据结果列名检查敏感字段是否可见：

```
1 if SENSITIVE_COL in df.columns:
2     st.warning(f"当前权限可见敏感属性 [{SENSITIVE_COL}]")
3 else:
4     st.success(f"安全验证通过：敏感属性 [{SENSITIVE_COL}] 已被视图机制屏蔽")
```

分析：该检查使权限差异在界面上更直观。管理员模式下结果包含 `freight_value`，因此显示敏感属性可见；分析师模式下结果来自视图，不包含该字段，因此显示脱敏通过。

6.4 越权访问测试

在分析师模式下，界面提供“强行查询基表”按钮。点击后，程序不再查询视图，而是直接尝试访问

`order_items`：

```
1  try:
2      conn = get_connection(role)
3      pd.read_sql(f"SELECT * FROM {BASE_TABLE} LIMIT 1;", conn)
4      st.warning("异常：分析师直接查询基表成功，请检查权限脚本是否已执行。")
5  except Exception as e:
6      st.error(
7          "访问控制生效！数据库内核拒绝了该请求:\n"
8          f"{format_database_error(e)}"
9      )
```

分析：这一按钮用于验证安全性是否真正由数据库保证。如果只是前端不提供基表查询入口，用户仍可能绕过界面直接发 SQL。现在即使应用主动提交越权 SQL，数据库也会返回权限拒绝错误，这表明 DAC 配置生效。

6.5 数据录入与触发器验证

管理员模式下提供订单项录入表单，默认构造一组满足外键约束的 `order_id` 和 `product_id`。提交时，Python 不判断 `freight_value > price`，而是直接执行插入：

```
1  cur.execute(
2      """
3      INSERT INTO order_items (
4          order_id,
5          order_item_id,
6          product_id,
7          price,
8          freight_value
9      )
10     VALUES (%s, %s, %s, %s, %s);
11     """,
12     (
13         order_id,
14         int(order_item_id),
15         product_id,
16         price,
17         freight_value,
18     ),
19 )
```

异常处理代码如下：

```

1 except psycopg2.DatabaseError as e:
2     if conn:
3         conn.rollback()
4     st.error(
5         "事务被中断 (触发器或约束拦截):\n"
6         f"{format_database_error(e)}"
7     )

```

分析：默认输入中 `price = 100`、`freight_value = 150`，会触发数据库异常。由于 Python 没有提前拦截该规则，界面显示的错误来源就是数据库触发器。这符合实验要求中“证明是数据库在进行拦截，而非 Python 代码”的要求。

7 运行与验证

7.1 初始化 SQL

执行初始化脚本：

```
1 | gsql -d ecommerce_db -U gaussdb -p 15432 -f demo_p4_security.sql
```

本机验证时使用 `psycogp2` 读取并执行同一 SQL 脚本，执行结果显示初始化成功：

```
1 | project4 init sql ok
```

7.2 触发器验证

验证脚本检查触发器是否存在：

```
1 | trigger_count= 1
```

尝试插入违规数据：

```

1 | price = 100
2 | freight_value = 150

```

数据库返回错误：

```
1 | ERROR:  数据完整性拦截: 订单项运费 150.00 不能高于商品售价 100.00
```

随后将 `freight_value` 改为不高于 `price`，合规插入可以通过，并在验证脚本中回滚事务，避免污染原始实验数据：

```
1 | valid_insert=accepted_then_rolled_back
```

分析：违规数据被触发器拦截，合规数据可以写入，说明触发器逻辑与预期一致。

7.3 权限验证

管理员查询 `order_items` 时，可以看到 `freight_value`：

```
1 | admin_sees_freight= True
```

分析师查询 `v_public_order_items` 时，结果列如下：

```

1 | analyst_view_columns= order_id,order_item_id,product_id,product_category_name,price
2 | analyst_view_hides_freight= True

```

分析师尝试直接查询物理表 `order_items` 时，数据库返回权限拒绝：

1 | ERROR: permission denied for relation order_items

分析：实验结果表明，分析师不仅在正常查询路径中看不到敏感字段，即使主动尝试绕过视图访问物理表，也会被数据库权限系统拒绝。

8 结果展示

8.1 管理员查询原始订单项

管理员点击“执行查询请求”后，界面展示来自 `order_items` 的订单项数据，其中包含敏感字段 `freight_value`。

当前会话身份：管理员 (Admin)

业务红线：订单项运费 `freight_value` 不能高于商品售价 `price`；敏感字段：`freight_value` 只允许管理员查看。

1. 数据安全性与视图机制测试

执行查询请求

执行 SQL: SELECT ... FROM order_items (物理基表)

	order_id	order_item_id	product_id	product_category_name	price	freight_value
0	ffe41c64501cc87c801fd61db3f6244	1	350688d9dc1e75f97be326363655e01	cama_mesa_banho	43	12.79
1	ffe18544fab9c95dfada21779c9644f	1	9c422a519119dca7575db5af1ba540e	informatica_acessorios	55.99	8.72
2	ffce4705a9662cd70adb13d4a31832d	1	72a30483855e2eafc67aee5dc2560482	esporte_lazer	99.9	16.95
3	ffcd46ef2263f404302a634eb57f7eb	1	32e07fd915822b0765e448c4dd74c828	informatica_acessorios	350	36.53
4	ffc94f6ce00a00581880bf54a75a037	1	4aa6014eceb682077f9dc4bffe05b0	utilidades_domesticas	299.99	43.41
5	ffbee3b5462987e66fb49b1c5411df2	1	6f0169f259bb0ff432bfff7d829b9946	casa_construcao	119.85	20.03
6	fffb9224b6fc7c43ebb0904318b10b5f	4	43423cdfde7fda63d0414ed38c11a73	relogios_presentes	55	34.19
7	fffb9224b6fc7c43ebb0904318b10b5f	3	43423cdfde7fda63d0414ed38c11a73	relogios_presentes	55	34.19
8	fffb9224b6fc7c43ebb0904318b10b5f	2	43423cdfde7fda63d0414ed38c11a73	relogios_presentes	55	34.19
9	fffb9224b6fc7c43ebb0904318b10b5f	1	43423cdfde7fda63d0414ed38c11a73	relogios_presentes	55	34.19

当前权限可见敏感属性 [freight_value]

8.2 分析师查询脱敏视图

分析师点击“执行查询请求”后，界面展示来自 `v_public_order_items` 的数据，字段包含 `order_id`、`order_item_id`、`product_id`、`product_category_name` 和 `price`，不包含 `freight_value`。

当前会话身份: 分析师 (Analyst)

业务红线: 订单项运费 `freight_value` 不能高于商品售价 `price`; 敏感字段: `freight_value` 只允许管理员查看。

1. 数据安全性与视图机制测试

执行查询请求

执行 SQL: SELECT * FROM v_public_order_items (脱敏视图)

	order_id	order_item_id	product_id	product_category_name	price	
0	fffe41c64501cc87c801fd61db3f6244		1	350688d9dc1e75ff97be326363655e01	cama_mesa_banho	43
1	fffe18544ffabc95dfada21779c9644f		1	9c422a519119dcad7575db5af1ba540e	informatica_acessorios	55.99
2	fffc4705a9662cd70adb13d4a31832d		1	72a30483855e2eafc67aee5dc2560482	esporte_lazer	99.9
3	fffd46ef2263f404302a634eb57f7eb		1	32e07fd915822b0765e448c4dd74c828	informatica_acessorios	350
4	fffc94fce00a00581880bf54a75a037		1	4aa6014eceb682077f9dc4bffe0c05b0	utilidades_domesticas	299.99
5	fffbec3b5462987e66fb49b1c5411df2		1	6f0169f259bb0ff432bfff7d829b9946	casa_construcao	119.85
6	fffb9224b6fc7c43ebb0904318b10b5f		4	43423cdfde7fda63d0414ed38c11a73	relogios_presentes	55
7	fffb9224b6fc7c43ebb0904318b10b5f		3	43423cdfde7fda63d0414ed38c11a73	relogios_presentes	55
8	fffb9224b6fc7c43ebb0904318b10b5f		2	43423cdfde7fda63d0414ed38c11a73	relogios_presentes	55
9	fffb9224b6fc7c43ebb0904318b10b5f		1	43423cdfde7fda63d0414ed38c11a73	relogios_presentes	55

安全验证通过: 敏感属性 [freight_value] 已被视图机制屏蔽

8.3 分析师越权访问基表

分析师点击“越权访问测试: 强行查询基表”后, 应用尝试执行 `SELECT * FROM order_items LIMIT 1`。数据库返回权限拒绝错误, 界面显示访问控制生效。

越权访问测试: 强行查询基表

访问控制生效! 数据库内核拒绝了该请求:
Execution failed on sql 'SELECT * FROM order_items
LIMIT 1;': permission denied for relation order_items
DETAIL: N/A

8.4 管理员录入违规订单项

管理员在录入表单中保持默认测试值 `price = 100`、`freight_value = 150`, 点击插入后触发器中断事务, 界面显示数据库返回的完整性错误。

2. 业务规则完整性与触发器测试

管理员录入订单项时, Python 不做业务拦截, 违规数据由数据库触发器中断。

订单 ID (order_id)

商品 ID (product_id)

00010242fe8c5a6d1ba2dd792cb16214

4244733e067ecb4970a6e2683c13e61

订单项序号 (order_item_id)

商品售价 price

运费 freight_value

2

- +

100.00

- +

150.00

- +

默认商品类别: cool_stuff

执行插入事务 (INSERT)

事务被中断 (触发器或约束拦截): ERROR: 数据完整性拦截: 订单项运费 150.00 不能高于商品售价 100.00

8.5 管理员录入合规订单项

管理员将 `freight_value` 调整为不高于 `price` 后再次提交，事务可以正常提交，界面显示数据符合完整性约束。

2. 业务规则完整性与触发器测试

管理员录入订单项时，Python 不做业务拦截，违规数据由数据库触发器中断。

订单 ID (order_id)

00010242fe8c5a6d1ba2dd792cb16214

商品 ID (product_id)

4244733e06e7ecb4970a6e2683c13e61

订单项序号 (order_item_id)

2

商品售价 price

100.00

运费 freight_value

1.00

默认商品类别: cool_stuff

执行插入事务 (INSERT)

事务提交成功: 数据符合完整性约束。

9 实验总结

本实验基于 Project1 构建的 Olist 电商 3NF 数据库，完成了数据库完整性控制和安全性控制两个方面的实现。在完整性控制部分，我选择 `order_items` 表中的 `price` 和 `freight_value` 设计复合业务规则，并通过触发器在插入和更新前进行检查，使违规订单项无法写入数据库。在安全性控制部分，我将 `freight_value` 作为敏感字段，通过 `v_public_order_items` 视图和 `analyst_user` 权限配置实现了分析师角色的数据脱敏访问。

通过 Streamlit 应用的角色切换与验证按钮，可以直观看到管理员和分析师在数据可见范围上的差异。分析师不仅在正常查询中无法看到 `freight_value`，在强行查询物理表时也会收到数据库返回的权限拒绝错误。触发器测试中，违规插入由数据库主动抛出异常，Python 只负责捕获并展示错误文本，这表明约束并不是依赖前端逻辑实现的。

总体而言，本实验加深了我对数据库内核层约束机制和自主存取控制的理解。触发器适合表达普通 `CHECK` 难以覆盖的业务语义，视图与权限授权则可以在不复制数据的前提下实现角色隔离。相比单纯在应用层判断，将关键规则放在数据库层能够更好地保证数据一致性与访问安全性。